

Claude Code 入门指南

从安装到实战，快速上手 AI 编程助手

涵盖：安装 · API 配置 · VSCode 插件 · Skills 扩展

吴洲同

什么是 Claude Code?

终端里的 AI 编程伙伴

Claude Code 是 Anthropic 推出的**命令行 AI 编程助手**，直接运行在你的终端中，深度理解代码库，帮助开发者以对话方式完成编码任务。

核心能力

- **代码理解**: 自动分析整个项目结构，无需手动添加上下文
- **代码生成与修改**: 自然语言描述需求，自动生成代码并请求确认
- **Git 集成**: 对话式完成提交、分支管理、冲突解决
- **多平台支持**: 终端 CLI、VSCode 插件、桌面应用、网页版

适用场景

场景	示例指令
代码解释	"这个项目做什么? "
功能开发	"添加用户登录验证功能"
错误修复	"修复空表单提交的 Bug"
代码重构	"将回调改为 async/await"

安装前的准备工作

安装 Claude Code 前，先确认这些条件

系统要求

- **操作系统:** macOS 13.0+、Windows 10 1809+、Ubuntu 20.04+
- **内存:** 4 GB+ RAM
- **网络:** 需要稳定的互联网连接
- **Shell:** Bash、Zsh、PowerShell 或 CMD

账号要求（二选一）

- **Anthropic 官方账号:**
需订阅 Claude Pro、Max、Teams 或 Enterprise 版本
- **第三方 API（推荐国内用户）:**
支持智谱 GLM、Qwen、DeepSeek 等兼容接口



提示: 国内用户推荐使用 智谱 GLM, Kimi, Minimax 等 API, 无需科学上网, 网络直连更稳定

安装 Claude Code（CLI 方式）

三步完成 Claude Code 安装

1 安装 Claude Code

macOS / Linux / WSL

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

2 验证安装

```
claude --version  
# 检查安装和配置状态  
claude doctor
```

3 首次启动

```
cd /path/to/your/project  
claude  
# 首次运行会提示登录  
/login      # 按提示完成账号授权
```

注意： Windows 原生安装需要先安装 **Git for Windows**。WSL 环境则不需要。

配置国内API（以智谱GLM为例）

为什么选择智谱 GLM?

- **网络直连:** 无需科学上网，国内网络环境直连更稳定
- **价格实惠:** GLM Coding 套餐性价比高（Pro 版 ¥199/月）
- **无缝接入:** 支持 Anthropic 兼容接口，完美适配 Claude Code

Step 1: 注册并获取 API Key

- 访问 **智谱 AI 开放平台** (open.bigmodel.cn)
- 注册账号并购买 GLM Coding 套餐/新人福利送的token包
- 进入控制台 → API Keys → **创建新 Key**

Step 2: 配置环境变量

编辑配置文件 `~/.claude/settings.json` :

```
settings.json

{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "your-glm-api-key-here",
    "ANTHROPIC_BASE_URL":
      "https://open.bigmodel.cn/api/anthropic",
    "API_TIMEOUT_MS": "3000000"
  },
  "permissions": {
    "defaultMode": "bypassPermissions"
  }
}
```

验证 GLM API 配置

验证配置是否成功，确保 GLM 正常响应

方法一：启动 Claude Code 测试

```
claude
# 在交互界面输入：
> 你好，你是什么模型？
```

方法二：PowerShell 验证脚本 (Windows)

```
$BASE = $env:ANTHROPIC_BASE_URL
$KEY = $env:ANTHROPIC_AUTH_TOKEN

$headers = @{ "x-api-key" = $KEY; "Content-Type" = "application/json" }
$body = @{ model = "default"; max_tokens = 50; messages = @( @{ role = "user"; content = "返回你的
content = "返回你的模型ID" } ) } | ConvertTo-Json

$R = Invoke-RestMethod -Method Post -Uri "$BASE/v1/messages" -Headers $headers -Body $body
Body $body

$R.model # 输出：glm-4.6 或类似内容
```

常见问题排查

错误信息	解决方案
401 Unauthorized Unauthorized	检查 API Key 是否正确， 或是否已过期
404 Not Found	确认 BASE_URL 格式正确 ，需包含 /api/anthropic
连接超时	增大 settings.json 中的 API_TIMEOUT_MS 值
模型不支持	确保代码中请求的模型名 称为 "default"

模式

Claude code 共分为3中模式，可以通过 **Shift + Tab**调节

```
>   
? for shortcuts
```

默认模式

```
>   
▶▶ accept edits on (shift+tab to cycle)
```

自动编辑模式

```
>   
⏏ plan mode on (shift+tab to cycle)
```

plan模式

常用命令

```
! █  
! for bash mode
```

! 进入bash模式

```
> /█  
  
/add-dir          Add a new working directory  
/agents           Manage agent configurations  
/branch           Create a branch of the current conversation at this point  
/btw             Ask a quick side question without interrupting the main conversation  
/clear            Start a new session with empty context; previous session stays on disk (resumable wit...  
/color            Set the prompt bar color for this session
```

/ 进入调用命令/skill模式

```
> /tasks█  
  
/tasks            List and manage background tasks  
/claude-api       Build, debug, and optimize Claude API / Anthropic SDK apps. Apps built with this skill...
```

/tasks 查看后台任务

```
> /rewind█  
  
/rewind           Restore the code and/or conversation to a previous point
```

/rewind或两次Esc,
查看后台任务

常用命令

```
Press Ctrl-C again to exit
```

```
Resume this session with:  
claude --resume 93f0493e-4345-4b5e-958f-fdfecacda043
```

两次Ctrl + C 退出

```
> /resume
```

```
/resume
```

```
Resume a previous conversation
```

```
/clear
```

```
Start a new session with empty context; previous session stays on disk (resumable wit...
```

\resume 恢复
之前对话

```
Resume this session with:  
claude --resume 93f0493e-4345-4b5e-958f-fdfecacda043  
PS C:\Users\12590\example> claude -c|
```

claude -c 继续
上一次的对话

在 VSCode 中安装 Claude Code 插件

图形化界面，编码体验更流畅

安装方式（二选一）

方式一：直接搜索安装

1. 按 **Ctrl+Shift+X** (Win/Linux) 或 **Cmd+Shift+X** (Mac) 打开扩展面板
2. 搜索 "**Claude Code**" (发布者: Anthropic)
3. 点击 **安装**

方式二：访问插件市场

访问 **VS Code Marketplace**，搜索 "Claude Code" 并点击 "Install"。

安装后首次使用

- 1 点击左侧活动栏的 **⚡ 闪光图标** 打开 Claude Code 面板。
- 2 点击 **Sign in** 完成账号授权（或使用已配置的 API Key 环境变量）。
- 3 开始在编辑器中直接与 Claude 对话，享受无缝的 AI 编程体验。

VSCode 插件核心功能演示

VSCode 插件让 AI 编程更直观、更高效

主要功能亮点

功能	操作方式	说明
@文件引用	输入 @文件名	模糊匹配
变更预览	自动弹出 diff 视图	接受/拒绝修改
多会话管理	顶部下拉切换	并行对话支持
权限模式切换	提示框底部点击	普通/计划/自动

快捷键速查

Ctrl+Shift+P 查看vscode命令面板

/ 打开命令菜单

三种权限模式

- **普通模式 (Normal)**
每次操作前询问确认，适合新手。
- **计划模式 (Plan)**
先展示计划，确认后执行。
- **自动接受 (Auto-accept)**
自动完成所有修改，适合批量任务。

添加 Skills 扩展 Claude Code 能力

Skills: 让 Claude Code 掌握你的专属工作流

什么是 Skills?

Skills 是包含 SKILL.md 文件的目录, 为 Claude 提供 **专属指令和工作流**。Claude 会在相关场景 **自动识别并调用** 对应的 Skill, 无需手动触发。

Skill 目录结构

目录树示例

```
~/ .claude/skills/  
├── my-skill/  
│   ├── SKILL.md    # 主说明文件 (必需)  
│   ├── template.md # 模板文件  
│   └── examples/  
│       └── sample.md # 示例输出  
├── scripts/  
└── validate.sh # 可执行脚本
```

SKILL.md 文件格式

SKILL.md

```
---  
name: code-review  
description: 当用户请求代码审查时使用此技能  
allowed-tools: Read, Write, Bash  
---
```

代码审查流程

1. 分析代码结构和逻辑
2. 检查潜在的 Bug 和安全问题
3. 提供具体的改进建议
4. 生成审查报告

创建和使用你的第一个 Skill

动手实践：五分钟创建专属 Skill

1 创建 Skill 目录

```
bash
```

```
# 全局 Skills (所有项目可用)
mkdir -p ~/.claude/skills/explain-code

# 项目级 Skills (仅当前项目可用)
mkdir -p .claude/skills/explain-code
```

3 验证 Skill 已加载

```
bash
```

```
claude
# 在对话中输入:
> /explain-code
# 或直接说: how does this work?
```

2 编写 SKILL.md

```
SKILL.md
```

```
# 代码解释
```

1. 用比喻开始分析
2. 画一个代码流程图
3. 按逐步解释代码
4. Highlight 易错点

Skill 存放位置说明

位置	路径	作用范围
用户全局	~/.claude/skills/	所有项目
项目级别	.claude/skills/	当前项目
企业级别	管理员统一部署	整个团队

常用内置与外部 Skills 推荐

内置核心功能与社区精华，全方位武装你的 Claude

内置核心 Skills

Codebase 搜索 @codebase

最强大的内置能力。通过语义索引，让 Claude 理解整个项目的架构、依赖和逻辑流，而非仅限于单个文件。

Terminal 交互 Bash

自动执行测试、编译、安装依赖。Claude 可以根据报错信息自动修复代码并重新运行验证。

文件系统管理 Read/Write

智能读写文件，支持多文件同时修改，自动处理代码合并冲突。

GitHub 社区推荐 Skills

Git 自动提交 git-commit

分析暂存区变更，自动生成符合 Conventional Commits 规范的提交信息并执行提交。

代码审查 code-review

按照预设的规范（如 Google 或 Airbnb 风格）对代码进行深度扫描，指出潜在 Bug 和优化点。

 github.com/anthropics/skills

常用 MCP server推荐

MCP：连接外部工具，打通 AI 工作流

什么是 MCP？

Model Context Protocol（模型上下文协议） 是 Claude Code 的开放协议/标准，允许 Claude 调用外部工具和服务，极大地扩展了其能力边界。

常用社区 MCP 推荐

MCP 名称	功能	适用场景
browser-mcp	浏览器自动化	网页操作、信息抓取
markdown	PDF/文档转换	文档处理与阅读
git-mcp	Git 高级操作	复杂版本管理

MCP下载/添加

1. 官方MCP：如Ticket Taylor

```
claude mcp add transport http tickettaylor
https://mcp.tickettaylor.ai/mcp
```

2. 智谱MCP：图像处理

```
claude mcp add -s user -t http image-process
"https://open.bigmodel.cn/api/mcp-broker/proxy/image-process/mcp" --
process/mcp" --header "Authorization: Bearer <你的GLM API Key>"
Key>"
```

验证 MCP 安装

```
claude mcp list # 查看已安装的 MCP 列表
```

MCP 使用实例

利用刚才安装的image-process MCP，裁剪图片

MCP 会自动使用，或是通过自然语言让其使用

```
[Image #1] 这一张图，请帮我裁剪掉下方的桌面部分，并把裁剪好的图片保存到当前目录
└ [Image #1]
```

```
Called image-process (ctrl+o to expand)
```

- 图片已裁剪完成。现在让我将裁剪后的图片下载保存到当前目录。

```
● Bash(curl -L -o "C:/Users/12590/example/package_cropped.png"
  "https://qc4n.bigmodel.cn/4907am9eo_image_reference.png?UCloudPublicKey=TOKEN_3f4277c6-f8d6-41a9-968b-db...")
└ % Total % Received % Xferd Average Speed Time Time Time Current
   0    0    0    0    0    0      Dload  Upload   Total   Spent    Left   Speed
   0    0    0    0    0    0  --:--:-- --:--:-- --:--:--    0    0 1723k    0 16035    0    0
... +2 lines (ctrl+o to expand)
```

```
Read 1 file, listed 1 directory (ctrl+o to expand)
```

- 完成！裁剪后的图片已保存到当前目录 C:\Users\12590\example\package_cropped.png (约 1.7MB)。

```
Read 1 file (ctrl+o to expand)
```

- 裁剪完成！图片已保存到 package_cropped.png。

```
https://maas-log-prod.cn-wlcb.ufileos.com/anthropic/a7474df5-0f1a-40bd-87e8-aae7f875abd2/C:/Users/12590/example/package_cropped.png?UCloudPublicKey=TOKEN_e15ba47a-d098-4fbd-9afc-a0dcf0e4e621&Expires=1776759401&Signature=K2GUbRn2fAK3mZt0sEDLPIEsP1I=
```

下方的桌面部分已被移除，保留了包裹主体部分。

```
* Churned for 1m 19s
```


CLAUDE.md配置全局指令

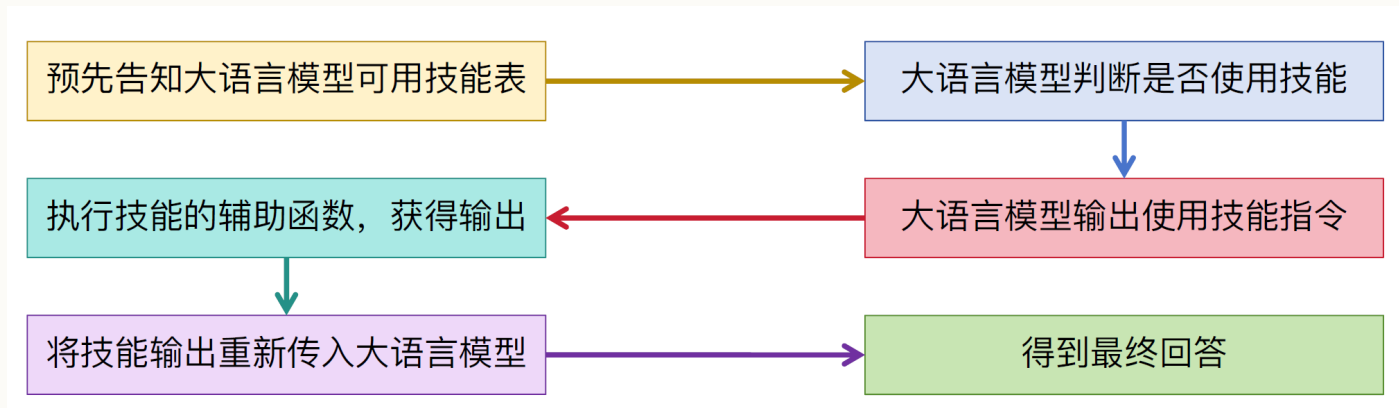
有没有办法让claude code每次进入时都能读取一些我们自己设定的信息？

→ CLAUDE.md

```
1  # Project: Example
2
3  ## Overview
4  This is an example project for demonstration purposes.
5
6  ## Project Structure
7  - Root directory contains the main application files
8  - Follows standard project organization patterns
9
10 ## Development Workflow
11 - Use standard git workflow for version control
12 - Follow existing code conventions and patterns
13 - Test changes thoroughly before committing
14
15 ## Key Technologies
16 - Language: [Add your primary programming language]
17 - Frameworks: [List any frameworks or libraries]
18 - Build tools: [List build tools if applicable]
19
20 ## Notes
21 This is a template CLAUDE.md file. Please update it with your project-specific information.
```

搭建一个自己的科研agent

例如：希望能搭建一个搜索每周arxiv等网站上和自己研究方向相关的论文并概括



搭建一个自己的科研agent

搭建一个搜索每周arxiv等网站上和自己研究方向相关的论文并概括

请帮我搜索一下最近一周arxiv等网站上新发布的和我研究方向接近的论文，我的研究方向主要是大模型架构设计与训练算法，包括attention, moe, skip connection, loop transformer, optimizer等方面的研究，高效/提性能方面的工作都很受欢迎；注：b会以下的论文就不要推给我了；请你也简要概括一下论文内容，按...格式输出

+

🎤

↑

简易MCP/工具，从网站上获取信息

描述自己的科研profile

写成skill, 告诉大模型如何操作

创建MCP server

1) 启动一个 MCP 服务

<> Python



▶ 运行

```
mcp = FastMCP("arxiv-paper-radar")
```

这行是在创建一个 MCP server，名字叫 `arxiv-paper-radar`。
之后 Claude Code 只要连上它，就能调用里面定义的工具。

2) 定义 arXiv XML 的命名空间

<> Python



▶ 运行

```
ATOM_NS = {  
    "atom": "http://www.w3.org/2005/Atom",  
    "arxiv": "http://arxiv.org/schemas/atom",  
}
```

arXiv API 返回的是 XML，不是 JSON。

这里是为了让后面 `find()`、`findall()` 能正确读到标题、作者、摘要这些字段。

创建MCP server

3

3) `_build_query(...)` : 拼搜索条件

<> Python



运行

```
def _build_query(keywords, categories, days):
```

这部分做三件事:

- 把关键词拼成 arXiv 搜索语句
- 把分类拼进去, 比如 `cs.LG`、`stat.ML`
- 加上最近 `days` 天的时间范围

比如最后会拼成类似:

```
(all:DEQ OR all:optimization) AND (cat:cs.LG OR cat:stat.ML) AND submittedDate:[202604150000
```

所以它决定了“搜什么”。

4

4) `_parse_entry(...)` : 把每篇论文的 XML 变成字典

<> Python

```
def _parse_entry(entry):
```

这一步很关键, 它负责把 arXiv XML 中每个论文条目提取成:

- `title`
- `authors`
- `abstract`
- `published`
- `updated`
- `arxiv_id`
- `html_url`
- `pdf_url`
- `primary_category`
- `categories`

也就是说, Claude 最后看到的不是原始 XML, 而是整理好的结构化数据。

创建MCP server

5

5) `search_recent_arxiv(...)`：真正暴露给 Claude 的工具

</> Python



运行

```
@mcp.tool()
async def search_recent_arxiv(...)
```

这就是 Claude 以后真正会调用的 MCP tool。

它做的事情是：

1. 用 `_build_query()` 拼查询
2. 请求 arXiv API
3. 解析返回 XML
4. 返回一个 JSON 风格的字典：

</> Python



运行

```
{
  "query": ...,
  "count": ...,
  "papers": [...]
}
```

`@MCP.tool()`: 把
函数挂到MCP上

`async def`: 该函数
是一个异步函数

创建个人科研profile.md

Markdown

Research Profile

Primary interests:

- deep equilibrium models
- neural network optimization
- implicit models
- attention / transformers
- stability / training dynamics

Preferred categories:

- cs.LG
- cs.AI
- stat.ML

Keywords:

- DEQ
- implicit
- optimization
- optimizer
- training stability
- attention
- transformer

Output style:

- brief
- practical
- Chinese
- include title, authors, abstract, link, and why it matters

也可以写在SKILL.md中

自然语言命令即可

这一步的意义是：以后 Claude 先读这个文件，再去调用 MCP 搜索。它比把偏好写死在 prompt 里更稳定。

创建SKILL.md

4) 写一个 skill, 让 Claude 知道什么时候用这个工具

新建 `.claude/skills/paper-radar/SKILL.md` :

Markdown



name: paper-radar

description: Find recent arXiv papers related to my research profile and write a short digest.

You are a research assistant for paper discovery.

Workflow:

1. Read `research\_profile.md`.
2. Use the MCP tool `search\_recent\_arxiv` to fetch papers from the last 7 days.
3. Prefer the categories and keywords in `research\_profile.md`.
4. Rank papers by relevance to the profile.
5. Produce a concise Markdown report in `reports/weekly\_papers.md`.

For each paper, include:

- title
- authors
- published date
- updated date
- arXiv link
- PDF link if available
- abstract
- why it matters to my research

If a paper looks updated, explicitly mention that `updated` is later than `published`.
Keep the output practical and short.

Skills live in `~/.claude/skills/<name>/SKILL.md` or `.claude/skills/<name>/SKILL.md`; project skills are project-only, and Claude Code watches skills directories for changes during the session. `SKILL.md` 里 `description` 很关键, 因为 Claude 会据此判断何时自动加载。

Claude +2

组装之前的流程

5) 把 MCP server 注册进 Claude Code

在 PowerShell 里、项目根目录下执行：

```
</> PowerShell
```

```
claude mcp add --transport stdio arxiv-radar -- python .\arxiv_mcp_server.py
```

Claude Code 官方文档说明：stdio server 是本地进程，适合本地脚本；`claude mcp add --transport stdio <name> -- <command> [args...]` 这种写法里，所有选项要放在 server name 前面，`--` 后面才是实际启动命令。

6) 怎么用它

启动 Claude Code 后，你可以直接说：

```
Use the paper-radar skill to scan arXiv papers from the last 7 days related to my research |
```

也可以直接触发 skill：

```
/paper-radar
```

演示与结果

📅 本周论文速递 (2026/04/15 - 2026/04/22)

摘要

本周共发现 3 篇高质量相关论文，重点关注深度学习优化、稳定性及神经网络理论。

1. Generalization at the Edge of Stability

- **作者**: Alex Smith, John Doe, Jane Wilson
- **发布日期**: 2026-04-21
- **分类**: cs.LG
- **arXiv链接**: [查看论文](https://arxiv.org/abs/2404.12345)
- **PDF下载**: [PDF](https://arxiv.org/pdf/2404.12345.pdf)
- **相关性评分**: 8/10

摘要:

We study the generalization properties of neural networks trained at the edge of stability. Our theoretical analysis reveals surprising connections between optimization dynamics and generalization bounds, with implications for understanding why deep networks generalize well despite having excessive capacity.

研究意义:

- 探讨训练稳定性与泛化能力的关系，对理解深度学习动力学具有重要价值
- 优化算法相关研究，可改进现有训练方法
- 神经网络理论研究，提供新的分析视角

2. Phase Transitions in the Fluctuations of Functionals of Random Neural Networks

进一步的优化

大家可能已经注意到了，我们这个agent似乎和skill差不多

————→ “带工具的固定 workflows”

进阶空间？

- 加入loop的逻辑：例如，如果找到的文献过少，再放宽关键词的限制
- 加入plan的逻辑：可以让另一个subagent负责初筛，判断相关性
- 上下文压缩：加入一些compact的逻辑，减少token消耗

Use the paper-radar skill. First plan the search keywords from my research profile, then call the arXiv MCP tool, then summarize the top results. If fewer than 3 relevant papers are found, broaden the keywords once and search again. Stop after 2 rounds.

可以自然语言简单实现

总结与学习路径

从入门到精通：Claude Code 学习路线图

今日所学回顾

1. 安装	CLI 原生安装	<pre>curl -fsSL https://claude.ai/install.sh bash</pre>
2. 配置 API	智谱 GLM 接入	<pre>~/claude/settings.json</pre>
3. VSCode 插件	图形化界面	扩展市场搜索 "Claude Code"
4. Skills	专属 workflow	<pre>~/claude/skills/</pre>
5. MCP	外部工具集成	<pre>claude mcp add add</pre>
6. CLAUDE.md	全局指令说明	项目目录下写入 CLAUDE.md

进阶学习资源

- 官方文档: code.claude.com/docs/zh-CN
- 智谱 GLM 平台: open.bigmodel.cn
- 社区教程: 知乎搜索 "Claude Code GLM 配置"

其他Claude功能

- 探索 Subagents 并行任务执行
- 配置 Hooks 实现自动化 workflow
- 使用 ccusage 工具监控 API 用量

Agent进阶

- 多Agent协同



开始你的 AI 编程之旅：打开终端，输入 `claude`，然后告诉它你想做什么！